Distributed Computing-An Introduction

Definitions

Early computing was performed on a single processor. A uni-processor, or monolithic computing, makes use of a single central processing unit (CPU) to execute one or more programs for each application.

A distributed system is a collection of independent computers, interconnected / via a network, that are capable of collaborating on a task. Computers are considered independent if they do not share memory or program execution space.

Such computers are called loosely coupled computers, as opposed to tightly coupled computers; the latter can share data using common memory space.

Distributed computing is computing performed in a distributed system.

• A network service is a service provided by a special kind of program known as a server on a network. The World Wide Web is such a service, as is electronic mail (email) and file transfer (FTP). A server program is just half of the story in the so-called client-server model of distributed computing.

• A network application is an application that runs on networked computers for end users. Network applications range from enterprise applications such as online shopping carts and electronic auction sites to noncommercial applications such as chat rooms and network games.

The distinction between network services and network applications is not always clear-cut, and the terms are often used interchangeably.

1.2 The History of Distributed Computing

In the beginning there were stand-alone computers, each of which was capable of executing stored programs. Connecting stand-alone computers so that data could be exchanged among them was a natural progression. Rudimentary connection of computers using cables for file sharing was practiced as early as the 1960s.

However, such an undertaking requires manual intervention and cannot be called a computer application when one or more computer programs execute autonomously to carry out a task. Such a computer application requires data communication, whereby two computers spontaneously exchange data using software and hardware in order to carry out the tasks inherent in the application.

The first Internet Request for Comments (RFC), RFC, is a proposal that specifies how participating hosts can exchange information with each other through messages. Whereas there may have been individual attempts to create network applications on a small scale the earliest network application was electronic mail, or email, by which the first message was reportedly sent in 1972 on a four-node ARPANET. (A node on a network is a computer, or host, that participates in the network.) Automated file transfer mechanisms, which allowed data files to be exchanged between hosts, were another natural progression, and as early as 1971 there was a proposal for such a mechanism. To this day, email and file transfer remain two of the most popular network services. The best-known network service, however, is undoubtedly the World Wide Web (WWW). The Web was originally conceived in the late

1980s by scientists at the Swiss research institute CERN in Geneva as an application that could support the access of hypertext over a network. The WWW has since become a platform for network applications and services, including email, search engines, and electronic commerce (e-commerce).

1.3 Different Forms of Computing

To understand what is meant by distributed computing in the context of this book, it is instructive to look at various forms of computing using computers.

Monolithic Computing

In the simplest form of computing, a single computer, such as a personal computer (PC), is used for computing. The computer is not connected to arty network, and thus it may use only those resources within its immediate access. This form of computing may be called monolithic computing. In the most basic monolithic computing, the computer is used by a single user at a time. The user runs applications on the system with no access to resources beyond those available with the system. When you use applications such as a word processing program or a spreadsheet on a PC, you are practicing this form of computing, which may be called *single-user monolithic computing.*

Multiple users can engage in monolithic computing. This form of computing where the resources of a single computer can be shared by concurrent users using a technique known as timesharing, was popular in the 1970s and 1980s. The computer that provides the centralized resource is usually called a *mainframe* to differentiate it from smaller computers such as minicomputers and microcomputers. Through devices known as *terminals,* users (who may be geographically dispersed) can be connected to the mainframe computer and interact with it during a terminal session.

Distributed Computing

In contrast distributed computing involves computing performed among and the network-connected computers, each of which has its own processor(s) their resources. A user, using a workstation, has full use of resources on the local computer to which its workstation is connected. In addition, through the interaction of the local computer and the remote computers, the user may access resources on the remote computers. The World Wide Web is an excellent example of this type of computing. When you use a browser to visit a Web site, a program such as Netscape or Internet Explorer runs on your local system and interacts with a program (known as a Web server) running on a remote system to fetch a file that may reside on yet another remote system.

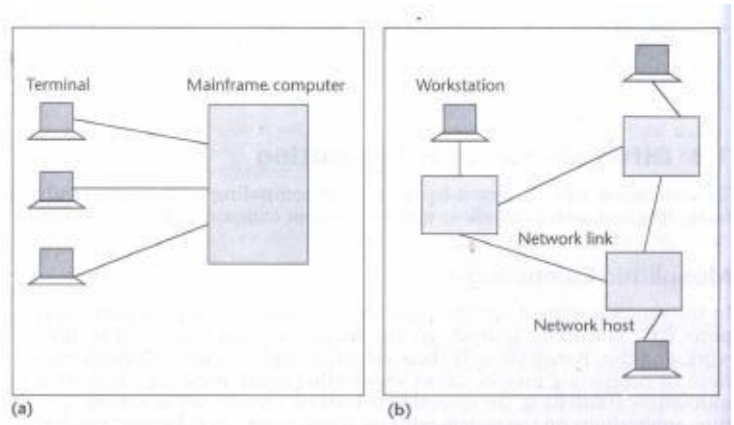


Figure 1.3 Centralized computing (a) versus distributed computing (b).

Parallel Computing

Similar to but distinct from distributed computing is a form of computing known as parallel computing or parallel processing, which uses more than one processor simultaneously to execute a single program. "Ideally, parallel processing makes a program run faster because there are more engines (CPUs) running it. In practice, it is often difficult to divide a program in such a way that separate CPUs can execute different portions without interfering with each other". Parallel computing is typically performed on a single computer that has multiple CPUs, but, it is also possible to perform parallel processing by connecting the computers in a network.

However, this type of parallel processing requires very sophisticated software called distributed processing software.

Using parallel computing, one can solve problems that are otherwise impossible solve on one computer or solve computing-intensive problems that are otherwise economically unfeasible.

Today, parallel computing is primarily used in large-scale scientific computing in areas such as biology, aerospace, weather forecasting, and semiconductor design. Although a fascinating subject, parallel computing is not within the scope of this book.

Cooperative Computing

Recently, the term *distributed computing* has also been applied to cooperative computing projects such as research for Extraterrestrial Intelligence (SETI). These are projects that parcel out large-scale computing to workstations on Internet hosts, making use of surplus cycles.

1.4 The Strengths and Weaknesses of Distributed Computing

Prior to the appearance of the World Wide Web, monolithic computing, such as business applications r1inning on a mainframe computer, or a single user using a personal computer to perform word processing or spreadsheet functions, was the only form of computing.

Distributed computing has become as important as-if not more important than-monolithic computing.

There are a number of **reasons for the popularity** of distributed computing:

The affordability of computers and availability of network access: Today's personal computer have computing power superior to that of the mainframe computers of the earlier days, at a fraction of the size and the cost. Coupled with the fact that connectivity to the Internet has become universally available and generally affordable, the large number of interconnected computers makes for an ideal Community for distributed computing.

Resource sharing: The architecture of distributed computing mirrors the computing architecture of modern organizations. Each organization independently maintain computers and resources that are local to the organization while sharing resources over the network. Using distributed computing organizations can share their resources very effectively. The Web, for example, is a powerful platform for sharing documents and other resources within and among organizations.

Scalability: With monolithic computing, the available resources are limited to the capacity of one computer. By contrast, distributed computing provides scalability in that increasing demand for resources can be addressed effectively with additional resources. For example, more computers providing a service such as email can be added to the network to satisfy an increase in the demand for that service.

Fault tolerance: Compared to monolithic computing, distributed computing provides the opportunity for fault tolerance in that a resource can be replicated (or mirrored) to sustain its availability in the presence of failures. For example, backup copies of a database can be maintained on different systems on the network, so that when one system fails, other copies can be accessed without disrupting the service. Although it is not possible to build a distributed system that is completely reliable in the presence of failures, it is the responsibility of a developer, when designing and implementing such a system, to maximize its fault tolerance. Fault tolerance in distributed computing is a complex topic that has received extensive attention in the research community.

In any form of computing, there is always a trade-off between advantages and disadvantages. The advantages already mentioned are offset by **disadvantages.**

Some of the most Significant ones are:

• ***Multiple points of failure:*** There are more points of failure in distributed computing. Since multiple computers are involved, all of which depend on the network for communication, the failure of one or more computers, or one or more network links, can spell trouble for a distributed computing system.

Security concerns: In a distributed system, there are more opportunities for unauthorized attack. Whereas in a centralized system all the computers and resources are typically under the control of a single administration, in a distributed system management is decentralized,

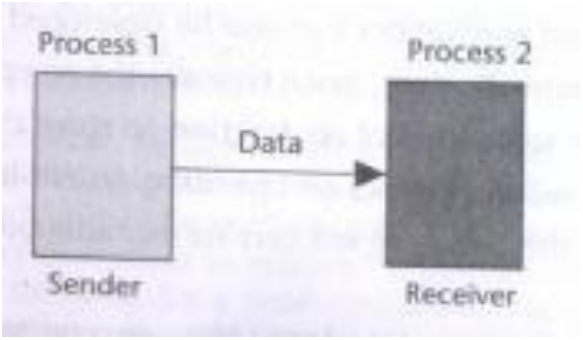
often involving a large number of independent organizations. The decentralization makes it difficult to implement and enforce security policies; hence distributed computing is vulnerable to security breaches and unauthorized access, which unfortunately can affect all participants on the system. This problem is clearly illustrated by well-known attacks on the Internet, such as worms and viruses.

Because of its importance, computer security is a widely researched and successful techniques have been developed for writing and deploying secure applications. Such techniques include encryption, keys, certificates, digital Signatures and boxes, authentication, and authorization.

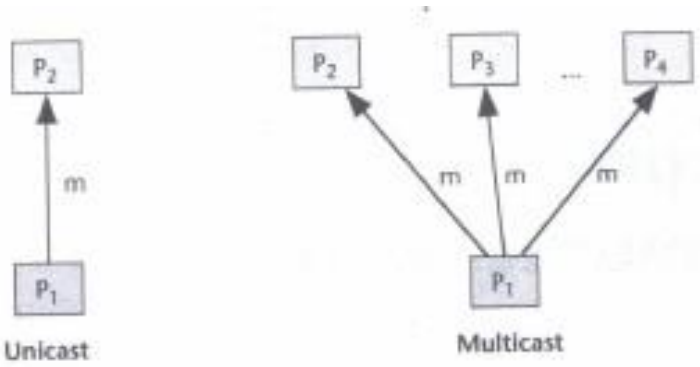
Interprocess Communications

The backbone of distributed computing is interprocess communications (lPC): the ability for separate, independent processes (as you will recall, *processes* are runtime representations of a program) to communicate among themselves to collaborate on a task. In this chapter, we will look at the fundamentals, the issues, the paradigms, and the implementations of IPC.

Two independent processes, possibly running on separate machines, exchange data over the interconnecting network.



In distributed computing, two or more processes engage in IPC in a protocol of rules that must be observed by the participants in data communication upon by the processes. A process may be a sender at some points during a protocol, a receiver at other points. When communication is from one process to a single other process, the IPC is said to be a unicast. When communication is from
one process to a group of processes, the IPC is said to be a multicast.



Modern-day operating systems such as UNIX and Windows provide facilities for interprocess communications. We will call these facilities operating system-level IPC facilities, to distinguish them from higher-level IPCs. System-level IPC facilities include message queues, semaphores, and shared memory. It is possible to develop network software using these system-level facilities directly. Examples of such programs are network device drivers and system evaluation programs.

Rudimentary distributed applications can also be developed using these facilities, although that is not normally done, since typically the complexities of the application require the use of some form of abstraction to spare the programmer of system- level details.

An IPC application program interface (API), also commonly expanded to application programming interface, provides an abstraction of the details and intricacies of the system-level facilities, thereby allowing the programmer to concentrate on the application logic; we will spend the rest of this chapter investigating IPC program interfaces.

1.1 An Archetypal IPC Program Interface

Consider a basic API that provides the minimum level of abstraction to facilitate IPC. Four primitive operations are needed. These operations are:

• **Send.** This operation is issued by a sending process for the purpose of transmitting data to a receiving process. The operation must allow the sending process to identify the receiving process and specify the data to be transmitted.

• **Receive.** This operation is issued by a receiving process for the purpose of accepting data from a sending process. The operation must allow the receiving process to identify the sending process and specify a memory space that allows the data to be stored, to be subsequently accessed by the receiver.

• **Connect.** For connection-oriented IPC, there must be operations that allow a logical connection to be established between the issuing process and a specified process; one process issues a request-to-connect (connect for short) operation while the other process issues an accept-connection operation.

• **Disconnect.** For connection-oriented IPC, this operation allows a previously established logical connection to be deallocated at both sides of the communication.

A process involved in IPC issues these operations in some predetermined order. The issuance of each operation causes the occurrence of an event. For example, a send operation issued by a sending process results in the event wherein data is transmitted to the receiving process, while a receive operation issued by a receiving process results in data being delivered to the process. Note that the participating processes issue operations independently, as each has no way of knowing the status of the other process) IPC operations are provided, implicitly or explicitly, in every distributed computing paradigm that we will study in subsequent chapters. Network service protocols can be implemented using primitive IPC operations. For example, in basic HTTP (Hypertext Transfer Protocol, used extensively on the World Wide Web) one process, a Web browser, issues a *connect* operation to establish a logical connection to another process, a Web server, followed by a *send* operation to the Web server which transmit s data representing a request. The Web server process in turn issues a *send* operation that transmits data requested by the Web browser process. At the end of the communication, each process issues a *disconnect* operation to terminate the connection.

1.2 Event Synchronization

A main difficulty with IPC is that the processes involved execute independently, with neither process knowing what takes place in the process at the other end. Consider basic HTTP, as described above.

The simplest way for an IPC facility to provide for event syncronisation is by using blocking, which is the suspension of the execution of an operation issued by the process has been completed.

To illustrate the use of blocking for event synchronization, consider basic HTTP. A browser process issues-a blocking *connect* operation, which blocks further execution of the process until the connection has been acknowledged by the server side.

Subsequently, the browser process issues a blocking receive operation which suspends execution of the process until the operation is completed. The blocking or unblocking is performed by the operating system and is initiated by the IPC facilities not by the programmer.

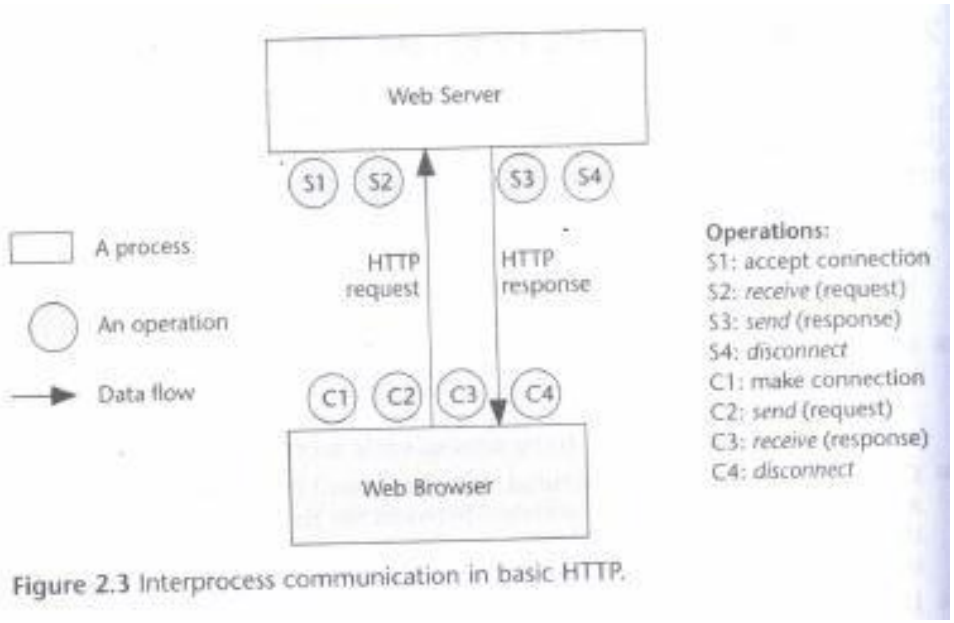
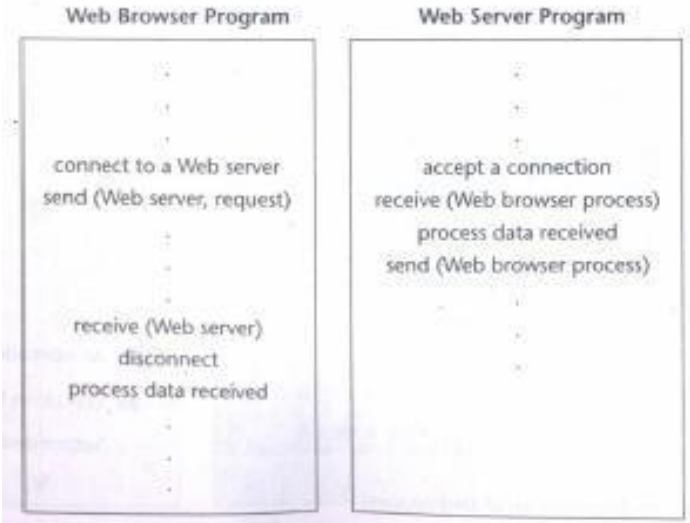


Figure 2.3 Interprocess communication in basic HTTP.

During execution, the process is suspended after each blocking operation is issued. The blocking is initiated by the IPC facility and the underlying operating systems when a blocking operation issued in a process is executed. The blocking will terminate subsequently when and if the operation is fulfilled, at which time the process is said to be *unblocked.* An unblocked process transits to the ready state and will resume execution in-time. In the event that the operation cannot be fulfilled, a blocked process will experience indefinite blocking, during which the process will remain in the blocked state indefinitely, unless intervening measures are taken.

Blocking operations are also referred to as **synchronous operations**. Alternatively, IPC operations may be **asynchronous or nonblocking operations**. An asynchronous operation issued by a process will not cause blocking, and therefore the process is free to continue with its execution once the asynchronous operation is issued to the IPC facility. The process will subsequently be notified by the IPC facility when and if the operation is fulfilled.



2.4 Program flow in two programs involved in IPC.

A nonblocking operation can be issued by a process when that process may proceed without waiting for the completion of the event that the operation initiates. For example, the *receive* operation issued by the Web browser must be blocking, because the browser process must wait for the response from the Web server in order to proceed with further processing. On the other hand, the *send* operation issued by the Web server can be nonblocking, because the Web server need not wait for the completion of the *send* operation before proceeding with the next operation (the *disconnect),* so that it may proceed to service other Web browser processes.

It is the programmer's responsibility to recognize the need for synchronization and to be cognizant of whether an operation is blocking. For example, if a nonblocking *receive* operation is used m coding the HTIP by an unwary programmer, and the data is assumed to have been received immediately after the issuance of the *receive* operation in the program, the subsequent processing may display invalid data or, worse yet, generate errors.

As the use of *send* and *receive* operations is fundamental of distributed computing.

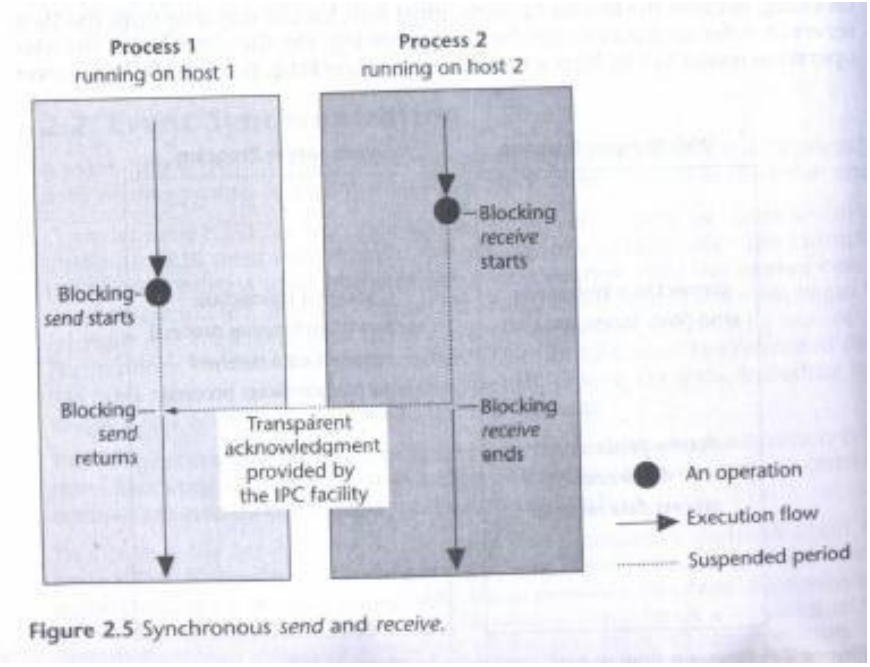


Figure 2.5 Synchronous *send* and *receive*.

Synchronous *Send* and Synchronous *Receive*

Event diagram illustrates the event synchronization for a protocol session implemented using synchronous *send* and *receive* operations. In this scenario, a receive operation issued causes the suspension of the issuing process (process2) until data is received to fulfill the operation. Likewise, a *send* operation issued causes the sending process (process 1) to suspend. When the data sent has been received by process 2, the IPC facility on host 2 sends an acknowledgment to the IPC facility on host 1, and process 1 may subsequently be unblocked.

The use of synchronous *send* and synchronous *receive* is warranted if the application logic of both processes requires that the data sent must be received before further processing can

proceed. Depending on the implementation of the IPC facility, the synchronous *receive* operation may not be fulfilled until the amount of data the receiver expects to receive has arrived.

Asynchronous *Send* and Synchronous *Receive*

Figure 2.6 illustrates an event diagram for a protocol session implemented using asynchronous *send* and synchronous *receive* operations. As before, a *receive* operation issued will cause the suspension of the issuing process until data is received to fulfill the operation.

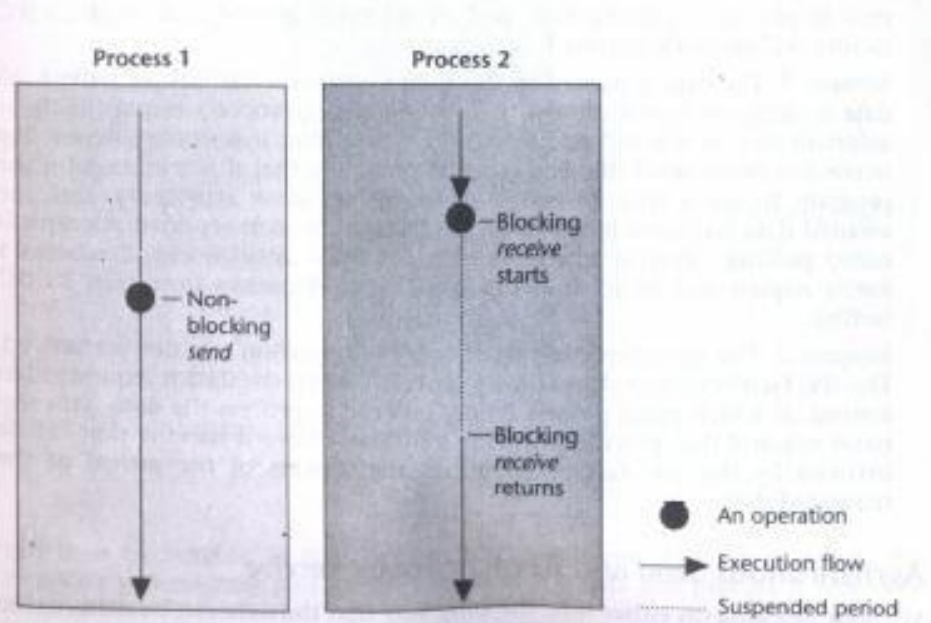


Figure 2.6 Asynchronous *send* and synchronous *receive*.

However, a *send* operation issued will not cause the sending process to suspend. In this case the sending process is never blocked, so no acknowledgment is necessary from the IPC facility on the host of process 2. This use of asynchronous *send* and synchronous *receive* is appropriate if the sender's application logic does not depend on the receiving of the data at the other end. However, depending on the implementation of the IPC facility, there is no guarantee that the data sent will actually be delivered to the receiver. For example, if the *send* operation is executed before the corresponding *receive* operation is issued on the other side, it is possible that the data will not be delivered to the receiving process unless the IPC facility makes provisions to retain the prematurely sent data.

Synchronous *Send* and Asynchronous *Receive*

Figure 2.7 illustrates different scenarios for a protocol session that employs synchronous *send* and asynchronous *receive* operations.

An asynchronous *receive* operation causes no blocking of the process that issues the operation, and the outcome will depend on the implementation of the IPC facility. The *receive* operation will, in all cases, return immediately, and there are three scenarios for what happens subsequently:

***Scenario* 1.**

The data requested by the receive operation has already arrived at the time when the *receive* operation is issued. In this case, the data is delivered to process 2 immediately, and an acknowledgment from host 2's IPC facility will unblock process l.

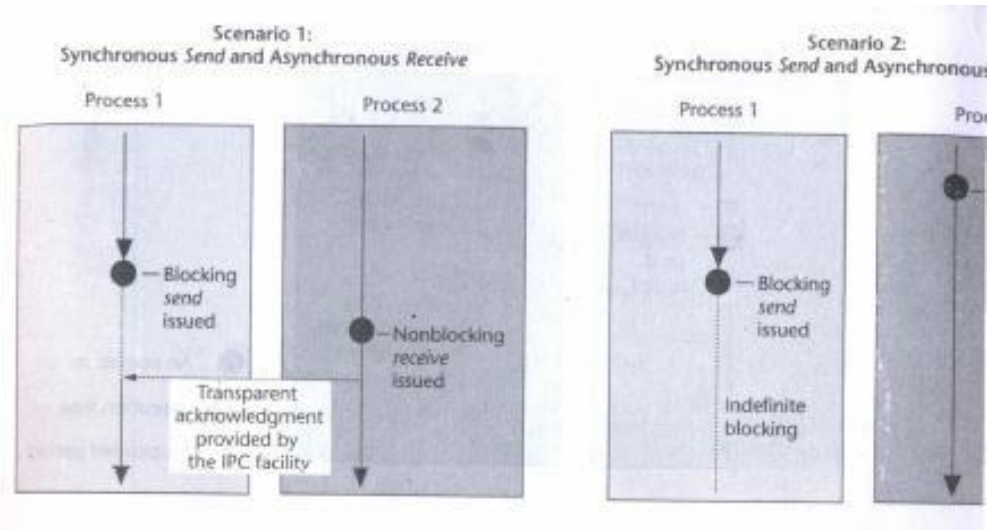
***Scenario* 2.**

The data requested by the *receive* operation has not yet arrived; no data is delivered to the process. It is the receiving process's responsibility to ascertain that it has indeed received the data and, if necessary, repeat the *receive* operation until the data has arrived. (Note that it is common for the program to use a loop to issue the *receive* operation repeatedly until the awaited data has been received. The technique of such repeated attempts is called polling.) Process 1 is blocked indefinitely until process 2 reissues a *receive* request and an acknowledgment eventually arrives from host 2's IPC facility.

***Scenario* 3.**

The data requested by the *receive* operation has not yet arrived. The IPC facility of host 2 will notify process 2 when the data it requested has arrived, at which point process 2 may proceed to process the data. This scenario requires that process 2 provide a listener or event handler that can be invoked by the IPe facility to notify the process of the arrival of the requested data.

Asynchronous *Send* and Asynchronous *Receive*



Without blocking on either side, the only way that the data can be delivered to the receiver is if the IPC facility retains the data received. The receiving process can then be notified of the data's arrival. Alternatively the receiving process may poll for the arrival of the data and process it when the awaited data has arrived.

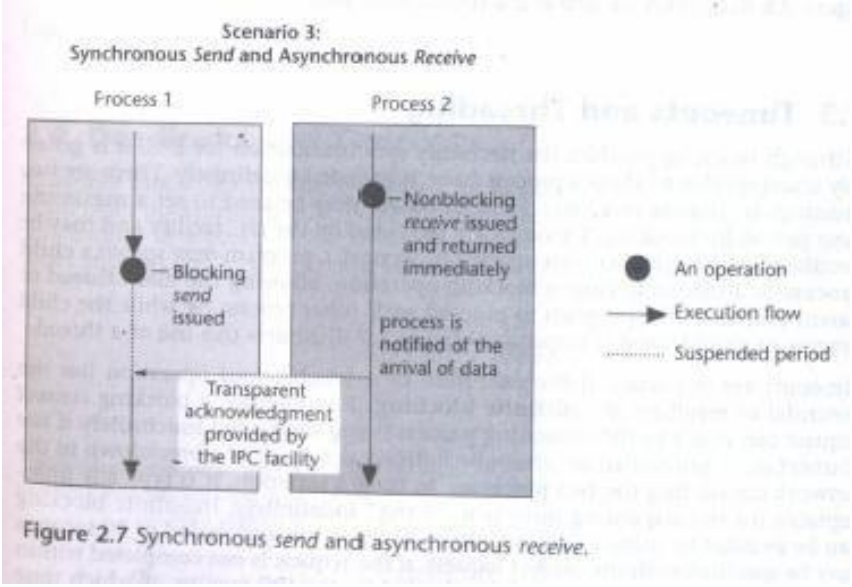


Figure 2.7 Synchronous *send* and asynchronous *receive*.

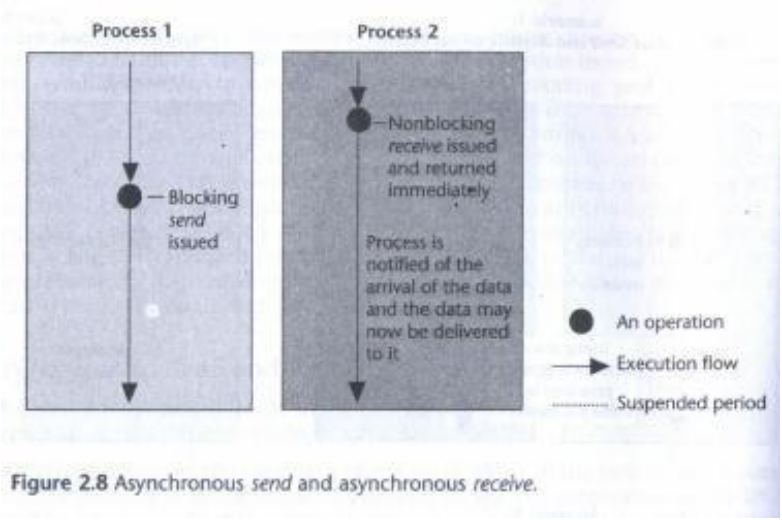


Figure 2.8 Asynchronous *send* and asynchronous *receive*.

1.3 Timeouts and Threading

Although blocking provides the necessary synchronization for IPC, it is generally unacceptable to allow a process to be suspended indefinitely. There are two measures to address this issue. First, timeouts may be used to set a maximum time period for blocking. Timeouts are provided by the IPC facility and may be specified in a program with an operation. Second, a program may spawn a child process or a thread to issue a blocking operation, allowing the main thread or parent process of the program to proceed with other processing while the child process or child thread is suspended.

Timeouts are important if the execution of a synchronous operation has the potential of resulting in indefinite blocking. For example, a blocking *connect* request can result in the requesting process being suspended indefinitely if the connection is unfulfilled or cannot be fulfilled as a result of a breakdown in the network connecting the two processes. In such a Situation, it is typically unacceptable for the requesting process to "hang" indefinitely. Indefinite blocking can be avoided by using a timeout. For example, a timeout period of 30 seconds may be specified with the *connect* request. If the request is not completed within approximately 30 seconds, it will be aborted by the IPC facility, at which time the requesting process will be unblocked, allowing it to resume processing.

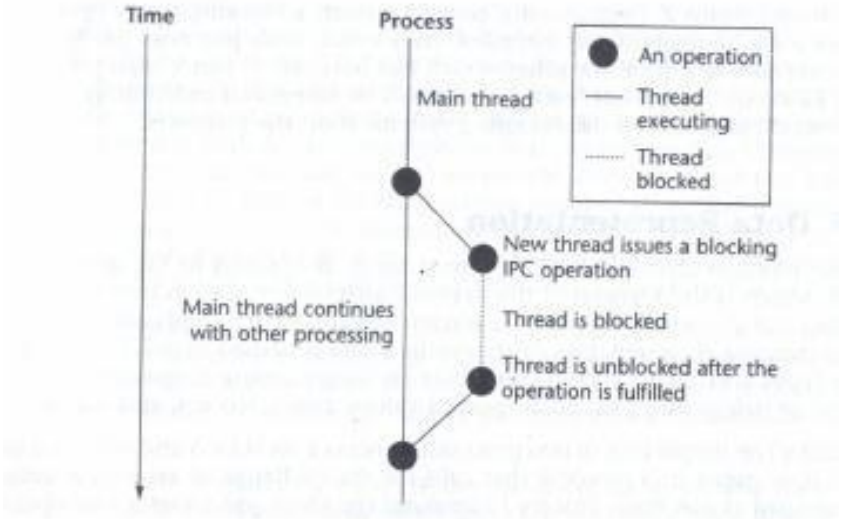


Figure 2.9 Using a thread for a blocking operation.

1.4 Deadlocks and Timeouts

Indefinite blocking may also be caused by a deadlock. In IPC, a deadlock can result from operations that were issued improperly, perhaps owing to a mis understanding of a protocol, or owing to programming errors.

In process 1, a blocking *receive* operation is issued to receive data from process 2. Concurrently process 2 issues a blocking receive operation where a send operation was intended. As a result both processes are blocked awaiting data sent from the other which can never occur. As a result each process will be suspended indefinitely until a timeout occurs or until the operating system aborts the process.

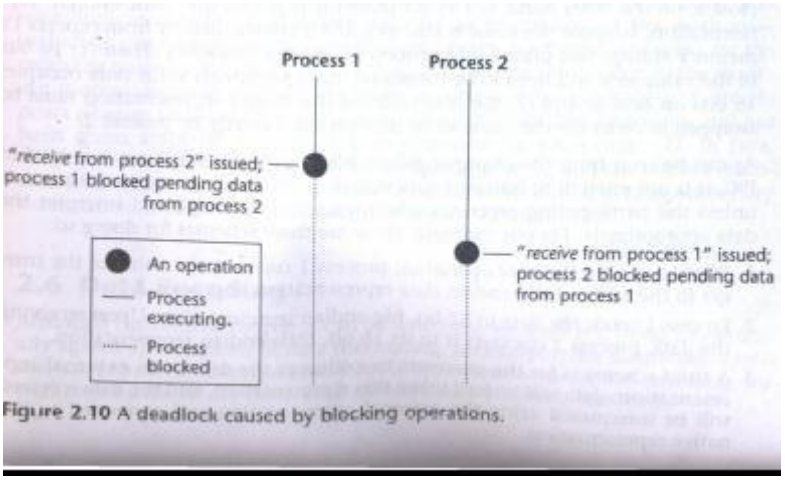


Figure 2.10 A deadlock caused by blocking operations.

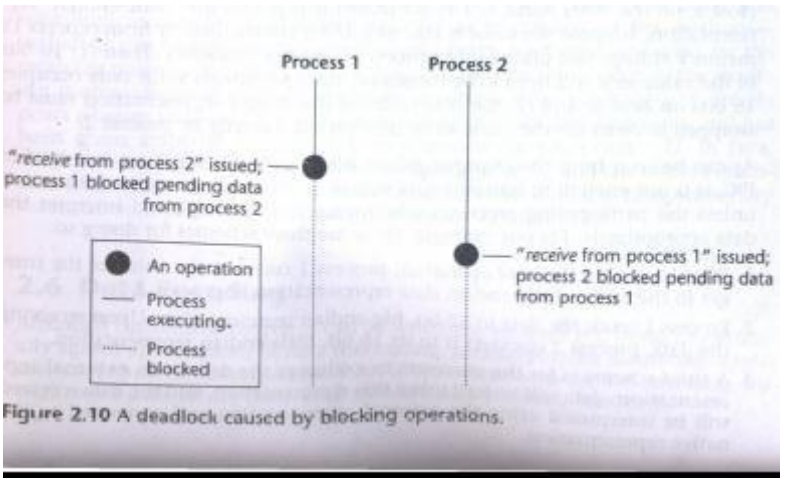


Figure 2.10 A deadlock caused by blocking operations.

1.5 Data Representation

At the physical layer (that is, the lowest layer, as opposed to the application layer, which is the highest) of the network architecture, data is transmitted as analog signals, which represent a binary stream. At the application layer, a more complex representation of transmitted data is needed in order to support data types and data structures provided in programming languages, such as character strings, integers, floating point values, arrays, records, and object.

Consider the simple case of two processes, process 1 on Host A and process 2 on host B, engaged in a protocol that calls for the exchange of an integer value determined at run time. Process 1 computes the value and issues a *send* operation to transmit it to process 2, which

issues a *receive* operation to accept the value, based on which process 2 performs further processing of the protocol.

1.6 Data Encoding

Although customized programs can be written to perform IPC using any mutual1y agreed upon scheme of data marshaling, general-purpose distributed applications require a universal, platform-independent scheme for encoding the exchanged data. Hence there exist network data encoding standards.

At the simplest level, an encoding scheme such as External Data Representation (XDR) allows a selected set of programming data types and data structures to be specified with IPC operations. The data marshaling and unmarshaling are performed automatically by the IPC facilities on the two ends, transparent to the programmer.

At a higher level of abstraction standards such as ASN.1 (Abstract Syntax Notation Number 1) exist. ASN.1 is an Open Systems Interconnection (OSI) standard that specifies a transfer syntax for representing network data. The standard covers a wide range of data structures (such as sets and sequences) and data types (such as integer, Boolean, and characters) and supports the concept of data tagging.

Each data item transmitted is encoded using syntax that specifies its type, its length, its value, and optionally a tag to identify a specific way of interpreting the syntax.

At an even higher level of abstraction, the Extensible Markup Language (XML) has emerged as a data description language for data sharing among applications, primarily Internet applications, using syntax similar to the Hypertext Markup Language (HTML), which is the language used for composing Web pages.

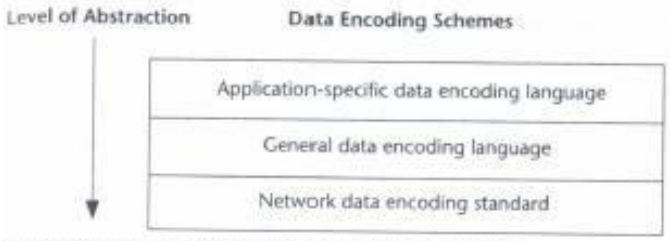


Figure 2.12 Network data representation standards.

1.7 Text-Based Protocols

Data marshaling is at its Simplest when the data exchanged is a stream of characters or text encoded using a representation such as ASCII. Exchanging data in text has the additional advantage that the data can be easily parsed in a program and displayed for human perusal. Hence it is a popular practice for protocols to exchange requests and responses in the form of character strings. Such protocols are said to be text-based. Many popular network protocols, including FTP

(File Transfer Protocol), HTTP, and SMTP (Simple Mail Transfer Protocol).

1.8 Request-Response Protocols

An Important type of protocol is the request-response protocol. In this protocol one side issues a request and awaits a response from the other Side.

Subsequently, another request may be issued, which in turn elicits another response. The protocol proceeds in an iteration of request-response, until the desired task is completed. The popular network protocols FTP, HTTP, and SMTP are all request-response protocols.

1.9 Event Diagram and Sequence Diagram

An event diagram, is a diagram .that can be used to document the detailed sequence of events and blocking during the execution of a protocol.

A solid line interval along the execution line represents a time period during which the process is active. A broken line interval represents when the process is blocked.

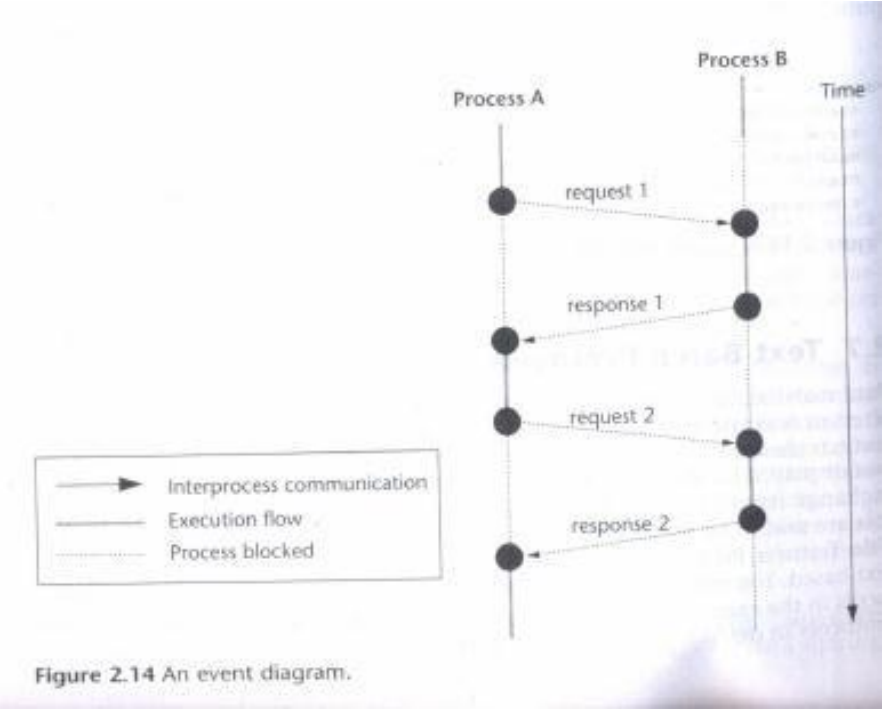


Figure 2.14 An event diagram.

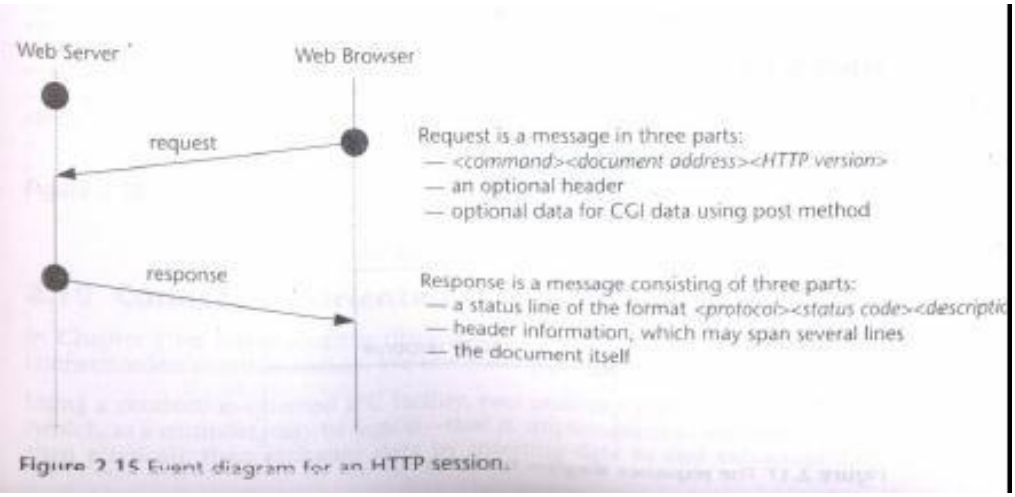


Figure 2.15 Event diagram for an HTTP session.

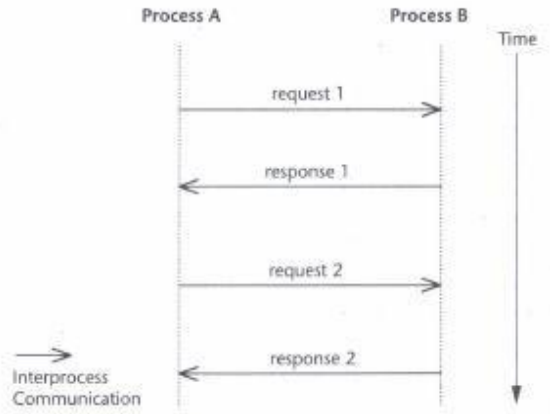


Figure 2.16 A sequence diagram.

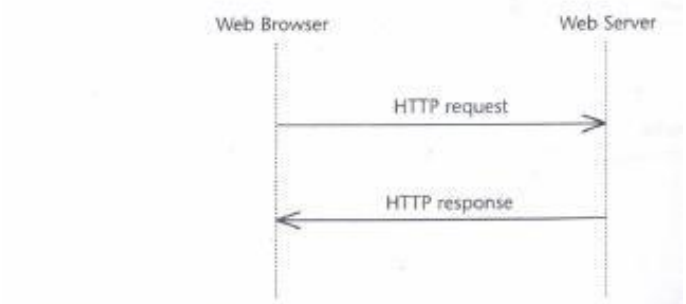


Figure 2.17 The sequence diagram for HTTP.